

03강

머신러닝

파이썬과 머신러닝

통계·데이터과학과 박준희



✧ 학습목차

- 1 모듈과 라이브러리
- 2 데이터 처리: Pandas
- 3 데이터 수치 분석: NumPy
- 4 데이터 시각화: Matplotlib



1

모듈과 라이브러리

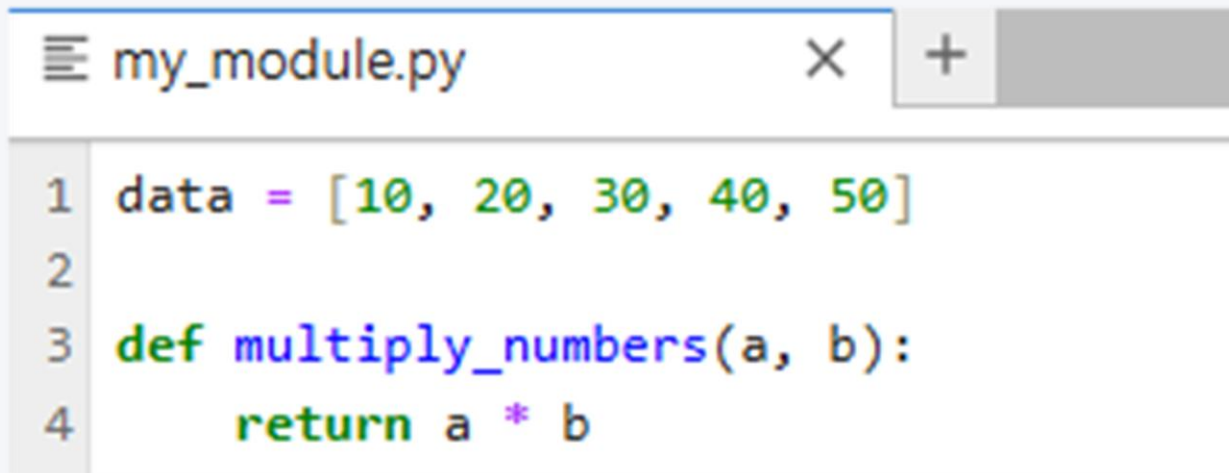


1

모듈과 라이브러리

● 모듈(Module)이란?

- (조립을 위한) 규격화된 부품
- 여러 함수, 데이터, 클래스 등을 하나의.py 파일에 저장한 것
- 모듈만들기: File -> New -> Python File



```

my_module.py
1 data = [10, 20, 30, 40, 50]
2
3 def multiply_numbers(a, b):
4     return a * b
    
```

1

모듈과 라이브러리

● 모듈(Module) 불러오기

- 불러오기 : `import 모듈`
- 짧게 불러오기 : `import 모듈 as 별명`
- 일부기능만 불러오기 : `from 모듈 import 기능`
- 모든기능 불러오기 : `from 모듈 import *`

1

모듈과 라이브러리

● 모듈(Module) 불러오기

```
[1]: import my_module

[2]: my_module.multiply_numbers(6,7)

[2]: 42

[3]: my_module.data

[3]: [10, 20, 30, 40, 50]
```

1

모듈과 라이브러리

● 모듈(Module) 불러오기

```
[4]: import my_module as mm
```

```
[5]: mm.multiply_numbers(3,7)
```

```
[5]: 21
```

```
[6]: mm.data
```

```
[6]: [10, 20, 30, 40, 50]
```

1

모듈과 라이브러리

● 모듈(Module) 불러오기

```
[7]: from my_module import multiply_numbers
```

```
[8]: multiply_numbers(10,10)
```

```
[8]: 100
```


1

모듈과 라이브러리

● 라이브러리(Library)란?

- 특정 기능을 수행하는 모듈 또는 모듈의 집합
- 표준 라이브러리: 파이썬에 내장된 라이브러리

예) os, sys, math, datetime

- 서드파티 라이브러리: 별도로 설치해야 하는 라이브러리

예) pandas, numpy, matplotlib

2

데이터 처리: Pandas



2

데이터 처리: Pandas

● 붓꽃(iris) 데이터



setosa

출처:Wikipedia



versicolor



virginica

2 데이터 처리: Pandas

● Pandas

- 'Panel Data' 에서 유래, 데이터 처리와 분석에 특화
- 프로젝트 폴더에 iris.csv 저장 후 Pandas 로 데이터 읽기

```
[12]: import pandas as pd
iris = pd.read_csv("iris.csv")
print(iris.head())
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

2 데이터 처리: Pandas

● Pandas

```
[13]: iris.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   Sepal.Length    150 non-null   float64
1   Sepal.Width     150 non-null   float64
2   Petal.Length    150 non-null   float64
3   Petal.Width     150 non-null   float64
4   Species         150 non-null   object  
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

2 데이터 처리: Pandas

● Pandas

```
[14]: iris.describe()
```

```
[14]:
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

2 데이터 처리: Pandas

● Pandas

```
[15]: print(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

[150 rows x 5 columns]

2 데이터 처리: Pandas

- **Pandas : 레이블 기반 인덱싱 (label-based indexing)**
 - 딕셔너리 스타일의 인덱싱

```
[48]: iris["Species"]

[48]: 0      setosa
      1      setosa
      2      setosa
      3      setosa
      4      setosa
      ...
     145  virginica
     146  virginica
     147  virginica
     148  virginica
     149  virginica
      Name: Species, Length: 150, dtype: object
```


2 데이터 처리: Pandas

● Pandas : 레이블 기반 인덱싱 (label-based indexing)

- 선택한 열에 value_counts 메서드 적용: 각 종의 데이터 수 확인

```
[54]: iris["Species"].value_counts()
```

```
[54]: Species
      setosa      50
      versicolor  50
      virginica   50
      Name: count, dtype: int64
```

2 데이터 처리: Pandas

● Pandas : iloc 메서드

- "integer-location" 의약자, 정수기반의 행과 열 위치 인덱싱

```
[37]: print(iris.head())
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
[38]: print(iris.iloc[0,0])
print(iris.iloc[1,2])
```

```
5.1
```

```
1.4
```

2 데이터 처리: Pandas

● Pandas : iloc 메서드

- "integer-location" 의약자, 정수기반의 행과 열 위치 인덱싱

```
[44]: print(iris.iloc[[0,2,4],1:4])
```

	Sepal.Width	Petal.Length	Petal.Width
0	3.5	1.4	0.2
2	3.2	1.3	0.2
4	3.6	1.4	0.2

2 데이터 처리: Pandas

● Pandas : 조건 필터링

- 특정 조건이나 기준에 따라 데이터를 분류하는 방법
- 참/거짓 조건을 사용하여 기준을 만족하는 행만 필터링

```
[58]: setosa = iris[iris['Species'] == 'setosa']
```

```
[67]: print(setosa)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
..	...	...	...	...	
47	4.6	3.2	1.4	0.2	setosa
48	5.3	3.7	1.5	0.2	setosa
49	5.0	3.3	1.4	0.2	setosa

2 데이터 처리: Pandas

● Pandas : 다중 조건 필터링

```
[72]: subset = iris[(iris['Sepal.Length'] > 6.5) & (iris['Species'] == 'versicolor')]
      print(subset)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
50	7.0	3.2	4.7	1.4	versicolor
52	6.9	3.1	4.9	1.5	versicolor
58	6.6	2.9	4.6	1.3	versicolor
65	6.7	3.1	4.4	1.4	versicolor
75	6.6	3.0	4.4	1.4	versicolor
76	6.8	2.8	4.8	1.4	versicolor
77	6.7	3.0	5.0	1.7	versicolor
86	6.7	3.1	4.7	1.5	versicolor

3

데이터 수치 분석: NumPy



3 데이터 수치 분석: NumPy

● NumPy

- 'Numerical Python'의 약어, 배열(array)의 수치 계산에 특화

```
[73]: import numpy as np
```

```
[92]: iris_num = iris.iloc[:, 0:4].values
```

```
[93]: np.mean(iris_num)
```

```
[93]: np.float64(3.4644999999999997)
```

```
[91]: np.mean?
```

Signature:

```
np.mean(  
    a,  
    axis=None,  
    dtype=None,  
    out=None,
```

⋮

3 데이터 수치 분석: NumPy

● NumPy

Examples

```
>>> import numpy as np
>>> a = np.array([[1, 2], [3, 4]])
>>> np.mean(a)
2.5
>>> np.mean(a, axis=0)
array([2., 3.])
>>> np.mean(a, axis=1)
array([1.5, 3.5])
```


3 데이터 수치 분석: NumPy

● NumPy: 각 특성(feature)에 대한 통계 수치 계산

```
[99]: np.mean(iris_num, axis=0)

[99]: array([5.84333333, 3.05733333, 3.758      , 1.19933333])

[100]: np.std(iris_num, axis=0)

[100]: array([0.82530129, 0.43441097, 1.75940407, 0.75969263])

[101]: np.min(iris_num, axis=0)

[101]: array([4.3, 2. , 1. , 0.1])

[102]: np.max(iris_num, axis=0)

[102]: array([7.9, 4.4, 6.9, 2.5])
```

출처:파이썬 프로그램 실습내용 중

3 데이터 수치 분석: NumPy

● NumPy: setosa 종의 각 특성 별 통계 수치를 계산하려면?

```
[111]: setosa = iris[iris['Species'] == 'setosa']
```

```
[112]: setosa_num = setosa.iloc[:, 0:4].values
```

```
[113]: np.mean(setosa_num, axis=0)
```

```
[113]: array([5.006, 3.428, 1.462, 0.246])
```

```
[114]: np.std(setosa_num, axis=0)
```

```
[114]: array([0.34894699, 0.37525458, 0.17191859, 0.10432641])
```

3 데이터 수치 분석: NumPy

● NumPy: 각 종의 특성 별 통계 수치를 계산하려면?

```
[116]: def stats(species):  
        data = iris[iris['Species'] == species]  
        data_num = data.iloc[:, 0:4].values  
        return np.mean(data_num, axis=0), np.std(data_num, axis=0)
```

```
[117]: stats("versicolor")
```

```
[117]: (array([5.936, 2.77 , 4.26 , 1.326]),  
        array([0.51098337, 0.31064449, 0.46518813, 0.19576517]))
```

```
[118]: stats("virginica")
```

```
[118]: (array([6.588, 2.974, 5.552, 2.026]),  
        array([0.62948868, 0.31925538, 0.54634787, 0.27188968]))
```

3 데이터 수치 분석: NumPy

●참고 : 그룹 별 각 특성들의 통계 수치 계산 (Pandas)

```
grouped_stats = iris.groupby('Species').agg(['mean', 'std', 'min', 'max'])
print(grouped_stats)
```

	Sepal.Length					Sepal.Width				
	mean	std	min	max		mean	std	min	max	\
Species										
setosa	5.006	0.352490	4.3	5.8		3.428	0.379064	2.3	4.4	
versicolor	5.936	0.516171	4.9	7.0		2.770	0.313798	2.0	3.4	
virginica	6.588	0.635880	4.9	7.9		2.974	0.322497	2.2	3.8	

	Petal.Length					Petal.Width				
	mean	std	min	max		mean	std	min	max	
Species										
setosa	1.462	0.173664	1.0	1.9		0.246	0.105386	0.1	0.6	
versicolor	4.260	0.469911	3.0	5.1		1.326	0.197753	1.0	1.8	
virginica	5.552	0.551895	4.5	6.9		2.026	0.274650	1.4	2.5	

3 데이터 수치 분석: NumPy

● NumPy : 상관 행렬(correlation matrix) 계산

```
[42]: iris_cor = np.corrcoef(iris_num, rowvar=False)
      print(iris_cor)

[[ 1.          -0.11756978  0.87175378  0.81794113]
 [-0.11756978  1.          -0.4284401  -0.36612593]
 [ 0.87175378 -0.4284401   1.          0.96286543]
 [ 0.81794113 -0.36612593  0.96286543  1.          ]]
```

```
[43]: setosa_cor = np.corrcoef(setosa_num, rowvar=False)
      print(setosa_cor)

[[1.          0.74254669 0.26717576 0.27809835]
 [0.74254669 1.          0.17769997 0.23275201]
 [0.26717576 0.17769997 1.          0.33163004]
 [0.27809835 0.23275201 0.33163004 1.          ]]
```


3 데이터 수치 분석: NumPy

● NumPy: 특성(feature) 값 전처리

- Standardization: 평균이 0, 분산이 1이 되도록 변환

```
[66]: iris_num = iris.iloc[:, 0:4]
iris_mean = np.mean(iris_num, axis=0)
iris_std = np.std(iris_num, axis=0)
iris_standard = (iris_num - iris_mean) / iris_std
print(iris_standard.head())
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
0	-0.900681	1.019004	-1.340227	-1.315444
1	-1.143017	-0.131979	-1.340227	-1.315444
2	-1.385353	0.328414	-1.397064	-1.315444
3	-1.506521	0.098217	-1.283389	-1.315444
4	-1.021849	1.249201	-1.340227	-1.315444

3 데이터 수치 분석: NumPy

● NumPy: 특성(feature) 값 전처리

- Standardization: 평균이 0, 분산이 1이 되도록 변환

```
[71]: print(iris_standard.agg(['mean', 'std', 'min', 'max']))
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
mean	-4.736952e-16	-7.815970e-16	-4.263256e-16	-4.736952e-16
std	1.003350e+00	1.003350e+00	1.003350e+00	1.003350e+00
min	-1.870024e+00	-2.433947e+00	-1.567576e+00	-1.447076e+00
max	2.492019e+00	3.090775e+00	1.785832e+00	1.712096e+00

3 데이터 수치 분석: NumPy

● NumPy : 특성(feature) 값 전처리

- Min-Max Scaling : 최소값이 0, 최대값이 1이 되도록 변환

```
[73]: iris_min = np.min(iris_num, axis=0)
iris_max = np.max(iris_num, axis=0)
iris_minmax = (iris_num - iris_min) / (iris_max - iris_min)
print(iris_minmax.head())
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
0	0.222222	0.625000	0.067797	0.041667
1	0.166667	0.416667	0.067797	0.041667
2	0.111111	0.500000	0.050847	0.041667
3	0.083333	0.458333	0.084746	0.041667
4	0.194444	0.666667	0.067797	0.041667

3 데이터 수치 분석: NumPy

● NumPy: 특성(feature) 값 전처리

- Min-Max Scaling: 최소값이 0, 최대값이 1이 되도록 변환

```
[75]: print(iris_minmax.agg(['mean', 'std', 'min', 'max']))
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
mean	0.428704	0.440556	0.467458	0.458056
std	0.230018	0.181611	0.299203	0.317599
min	0.000000	0.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000	1.000000

3 데이터 수치 분석: NumPy

● NumPy: 주성분 분석(principal component analysis)

```
[107]: covariance_matrix = np.cov(iris_standard, rowvar=False)
       eigen_values, eigen_vectors = np.linalg.eig(covariance_matrix)
       print(eigen_values)
       print(eigen_vectors)
```

```
[2.93808505 0.9201649  0.14774182 0.02085386]
[[ 0.52106591 -0.37741762 -0.71956635  0.26128628]
 [-0.26934744 -0.92329566  0.24438178 -0.12350962]
 [ 0.5804131  -0.02449161  0.14212637 -0.80144925]
 [ 0.56485654 -0.06694199  0.63427274  0.52359713]]
```

3 데이터 수치 분석: NumPy

● NumPy : 주성분 분석(principal component analysis)

```
[102]: pc_4 = eigen_vectors
transformed_4 = np.dot(iris_standard, pc_4)
print(transformed_4[:7])
```

```
[[-2.26470281 -0.4800266 -0.12770602  0.0241682 ]
 [-2.08096115  0.67413356 -0.23460885  0.10300677]
 [-2.36422905  0.34190802  0.04420148  0.02837705]
 [-2.29938422  0.59739451  0.09129011 -0.06595556]
 [-2.38984217 -0.64683538  0.0157382 -0.03592281]
 [-2.07563095 -1.48917752  0.02696829  0.00660818]
 [-2.44402884 -0.0476442  0.3354704 -0.03677556]]
```

3 데이터 수치 분석: NumPy

● NumPy: 주성분 분석(principal component analysis)

```
[106]: tf_4_pd = pd.DataFrame(transformed_4, columns=['PC1', 'PC2', 'PC3', 'PC4'])
print(tf_4_pd.agg(['mean', 'std', 'min', 'max']))
```

	PC1	PC2	PC3	PC4
mean	-1.894781e-16	9.651539e-16	-1.480297e-16	2.072416e-17
std	1.714084e+00	9.592523e-01	3.843720e-01	1.444087e-01
min	-2.774345e+00	-2.686284e+00	-8.574218e-01	-4.894835e-01
max	3.310696e+00	2.654073e+00	1.005401e+00	4.696966e-01

```
[108]: print(np.corrcoef(tf_4_pd, rowvar=False))
```

[	1.00000000e+00	2.03019129e-16	3.74341563e-16	-1.09572082e-15]
[	2.03019129e-16	1.00000000e+00	-1.29323356e-15	2.28605571e-16]
[	3.74341563e-16	-1.29323356e-15	1.00000000e+00	8.72554062e-16]
[	-1.09572082e-15	2.28605571e-16	8.72554062e-16	1.00000000e+00]

3 데이터 수치 분석: NumPy

● NumPy : 주성분 분석(principal component analysis)

```
[103]: pc_2 = eigen_vectors[:, :2]
transformed_2 = np.dot(iris_standard, pc_2)
print(transformed_2[:7])
```

```
[[-2.26470281 -0.4800266 ]
 [-2.08096115  0.67413356]
 [-2.36422905  0.34190802]
 [-2.29938422  0.59739451]
 [-2.38984217 -0.64683538]
 [-2.07563095 -1.48917752]
 [-2.44402884 -0.0476442 ]]
```

3 데이터 수치 분석: NumPy

- NumPy: 주성분 분석(principal component analysis)

```
[109]: print(np.corrcoef(transformed_2, rowvar=False))  
  
[[1.00000000e+00 2.03019129e-16]  
 [2.03019129e-16 1.00000000e+00]]
```

4

데이터 시각화: Matplotlib



4 데이터 시각화: Matplotlib

● Matplotlib

- MATLAB의 plot 기능에서 유래 / 데이터 시각화에 특화

```
[125]: import matplotlib.pyplot as plt

       setosa = iris[iris['Species'] == 'setosa']
       versicolor = iris[iris['Species'] == 'versicolor']
       virginica = iris[iris['Species'] == 'virginica']
```


4 데이터 시각화: Matplotlib

● Matplotlib

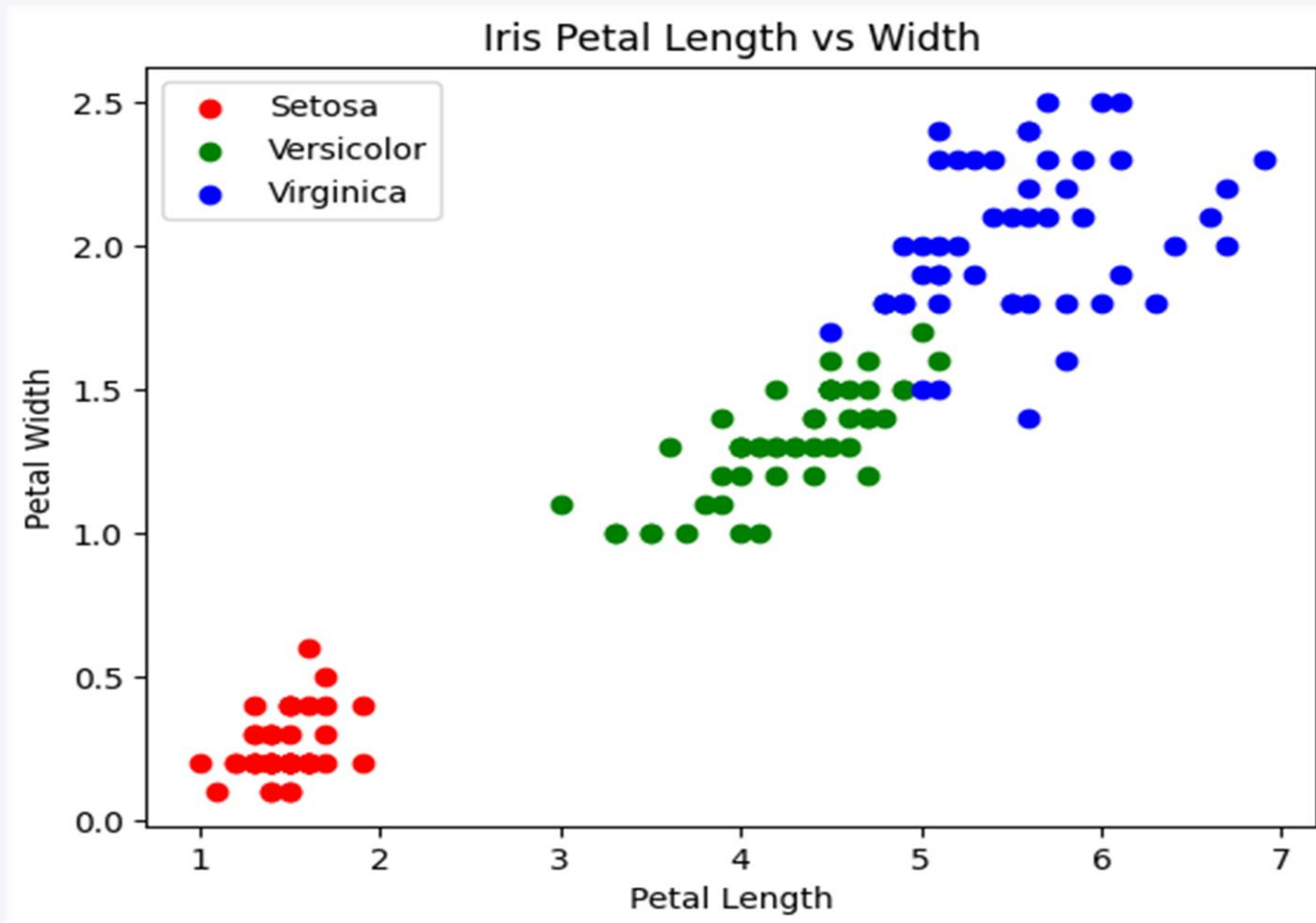
```
[27]: plt.scatter(setosa['Petal.Length'], setosa['Petal.Width'],
                color='red', label='Setosa')
plt.scatter(versicolor['Petal.Length'], versicolor['Petal.Width'],
            color='green', label='Versicolor')
plt.scatter(virginica['Petal.Length'], virginica['Petal.Width'],
            color='blue', label='Virginica')

plt.title('Iris Petal Length vs Width')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')

plt.legend()
plt.show()
```

4 데이터 시각화: Matplotlib

● Matplotlib



출처: 파이썬 프로그램 실습 내용 중

★ 정리하기

- 모듈

함수, 데이터 등을 저장하는 단일 py파일

- 라이브러리

특정 기능을 제공하는 모듈 또는 모듈의 집합

- Pandas

데이터를 불러오고, 인덱싱과 필터링 등으로 효율적 처리 가능

- NumPy

데이터의 (고성능) 수치 계산 가능

- Matplotlib

데이터 시각화 가능

04강

다음시간안내

확률 분포와 랜덤 벡터 기초

